Assignment 1: A Pentium M style hybrid branch predictor

COSC 6385 Computer Architecture, Fall 2026

Bryan E. Tuck
University of Houston
https://github.com/Bryan-Tuck/cosc6385-hw1-branch-predictor

October 2026

1 Introduction

The Pentium M predicts branch direction with a bimodal table, a global predictor and a loop predictor [3]. This assignment covers the first two. The bimodal table of 2-bit counters, indexed by the branch address, holds each branch's usual direction. The global predictor, a set-associative table of tagged 2-bit counters, holds directions that depend on the path taken to the branch. The Pentium M indexes it with the branch address XORed with a path information register (PIR). The assignment does not require the PIR, so my predictor, like its Appendix B example, XORs the address with a global history register (GHR) of conditional branch outcomes instead. When a set holds an entry whose tag matches, the global counter is used; otherwise the bimodal counter is used and the global predictor allocates an entry for the next encounter.

I implemented this hybrid as the `pm_predictor` class of `my_predictor.h` in the CBP2 branch prediction infrastructure [1], following the structure and the worked example in the assignment [2], and measured it on the 20 CBP2 traces. It averages 7.759 mispredictions per thousand instructions (MPKI) over the traces (Section 3). The loop predictor and branch target prediction are outside the assignment and are not implemented, and I did not attempt the extra credit `cpm_predictor`. The predictor uses no third-party cache code.

2 Predictor design

Figure 1 shows the structure the code implements. The sizes follow the assignment figure: a 4096-entry bimodal table, a 4-way global predictor with 512 sets, and Index = HASH[14:6], Tag = HASH[5:0]. Each size is a `#define` at the top of the `pm_predictor` section of `my_predictor.h`.

2.1 Bimodal table

The bimodal table holds $2^{12} = 4096$ saturating 2-bit counters. For a conditional branch at address A the counter index is the low 12 bits,

$$i_{\text{bim}} = A \bmod 2^{12}, \tag{1}$$

and the prediction is the counter's upper bit (states 2 and 3 predict taken). The counters start at 0. The index is IP[11:0], as in the assignment figure. The Appendix B example drops the two low bits of its word-aligned addresses, but x86 branch instructions can start at any byte, so the index keeps bits [1:0].

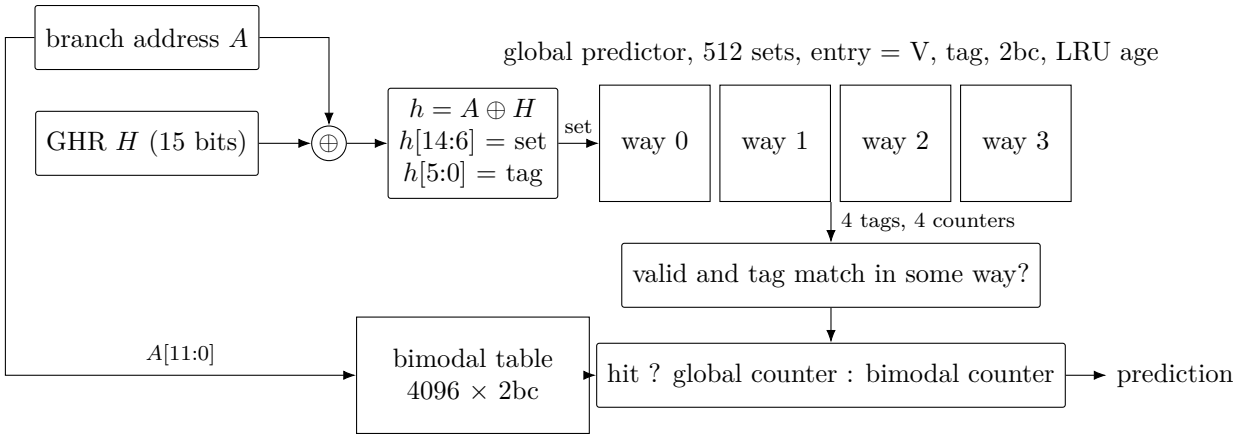


Figure 1: Structure of `pm_predictor`. The hash h of the branch address and the global history register selects a set of the 4-way global predictor and supplies the tag; the bimodal table is indexed by the low address bits. A tag hit selects the global prediction, a miss selects the bimodal prediction.

2.2 Global predictor

The global predictor has four ways of 512 sets. Each entry, a `pm_global_entry` in the code, stores a valid bit, a 6-bit tag, a 2-bit counter and a 2-bit LRU age. The global history register H holds the outcomes of the last 15 conditional branches, newest in bit 0. The lookup hashes the address with the history and splits the hash into a set index and a tag:

$$h = (A \oplus H) \bmod 2^{15}, \tag{2}$$

$$s = \lfloor h/2^6 \rfloor = h[14{:}6], \qquad t = h \bmod 2^6 = h[5{:}0]. \tag{3}$$

The upper hash bits select the set and the lower bits form the tag, following the labels Index = HASH[14:6] and Tag = HASH[5:0] in the assignment figure and the row/tag split of its Appendix B example, where a 4-bit hash gives the row from bits [3:2] and the tag from bits [1:0]. Because the 15-bit history covers the whole hash, the tag depends on the six most recent outcomes XORed with address bits [5:0], and the set index on the older nine outcomes XORed with address bits [14:6].

The lookup compares t with the tag of each valid way in set s. A match is a hit and the global prediction is the upper bit of that way's counter. On a miss the lookup records the LRU way of the set (age = 3) as the victim for allocation. Replacement is true LRU kept with per-entry ages. When a way is used, every way of the set whose age was smaller than the used way's age grows by one and the used way's age becomes 0. In the constructor, way w gets age $3 - w$, so an empty set fills way 0, 1, 2, 3 in that order and no valid entry is evicted while an invalid way remains. For two ways the age is a single bit that is 1 in the LRU way, which is the LRU column of the Appendix B tables.

2.3 Selection and update

The final prediction is the global prediction on a hit and the bimodal prediction on a miss. There is no chooser table; the tag comparison selects the table. The predictor returns a `pm_update` object that records the bimodal index, the set, the tag, the hit flag and the way, so `update` does not repeat the lookup.

After the driver reports the outcome, `update` changes state only for conditional branches:

1. The bimodal counter at i_{bim} is trained toward the outcome whether the global lookup hit or missed.

2. On a hit the matching global counter is trained toward the outcome. On a miss the LRU way is allocated. Its valid bit is set, its tag becomes t, and its existing counter is trained toward the outcome without being reset, as in the Appendix B example, where in step 9 the evicted entry's counter 3 becomes 2 after a not-taken outcome.

3. The used way becomes most recently used, as described above.

4. The outcome is shifted into bit 0 of H, which is masked to 15 bits.

Unconditional branches are predicted taken and do not touch the tables or the history, the same convention as the sample `gshare_predictor` that ships with the infrastructure. Branch targets are not predicted; `predict` sets the target prediction to 0, as the starter code does.

3 Results

Each CBP2 trace holds the branches of 100 million instructions of one SPEC CPU2000 or SPECjvm98 program. The driver counts direction mispredictions of conditional branches and reports MPKI, mispredictions per thousand instructions; lower is better. Table 1 lists the result for every trace. The raw numbers are in `results/hybrid.csv`.

Table 1: MPKI of `pm_predictor` per trace (lower is better).

Trace	MPKI	Trace	MPKI
164.gzip	12.697	213.javac	3.001
175.vpr	15.969	222.mpegaudio	2.932
176.gcc	18.009	227.mtrt	3.453
181.mcf	18.269	228.jack	4.239
186.crafty	9.109	252.eon	2.153
197.parser	10.936	253.perlbmk	5.095
201.compress	8.019	254.gap	4.309
202.jess	2.005	255.vortex	1.643
205.raytrace	3.602	256.bzip2	0.115
209.db	4.801	300.twolf	24.821
Average over the 20 traces			7.759

MPKI ranges from 0.115 on bzip2 to 24.821 on twolf. The four highest, twolf, mcf, gcc and vpr, are all SPECint programs.

References

[1] Daniel A. Jiménez. Infrastructure for the branch prediction competition (CBP2), version 2. Distributed with the course as `cbp2-infrastructure-v2`, 2006.

[2] Weidong Shi. Assignment 1: Branch predictor implementation. COSC 6385 Computer Architecture, University of Houston, 2026.

[3] Vladimir Uzelac and Aleksandar Milenković. Experiment flows and microbenchmarks for reverse engineering of branch predictor structures. In *IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 207–217, 2009.